P.Vishwa Nandan(192473015)

## TASK 3 – FROM SKETCH TO LAYOUT (9 Marks)

### 1. Aim

To design and implement a campus library search screen using Java Swing layout managers. The application must organise the search field, category selection, results area, navigation buttons, and status message.

The screen must also adjust properly when the window is resized.

### 2. Problem Description

A campus library requires a graphical search screen with the following components:

1. A title at the top.
2. A search row containing a text field, category selection, and Search button.
3. A results area in the centre.
4. Prev, Next, and Reserve buttons at the bottom.
5. A status message.

When the window is resized, the results area must grow, while the search row and buttons maintain their preferred sizes.

### 3. Task 3.1 – Choose the Layout Managers (3 Marks)

Java Swing provides different layout managers to organise GUI components.

The following four layout managers are used in this design.

#### 3.1 BorderLayout

**Purpose:** Divides a container into five regions: NORTH, SOUTH, EAST, WEST, and CENTER.

**Usage:** Used for the main application window.

- NORTH – Title and search section.
- CENTER – Library search results.
- SOUTH – Navigation buttons and status message.

**Justification:** BorderLayout allows the centre results area to occupy the remaining available space and grow when the window is resized.

#### 3.2 FlowLayout

**Purpose:** Arranges components from left to right in a row.

**Usage:** Used for the title section, search row, and navigation button row.

**Justification:** FlowLayout keeps the search controls and buttons organised without unnecessarily stretching them when the window size changes.

### 3.3 GridLayout

**Purpose:** Arranges components in rows and columns of equal-sized cells.

**Usage:** Used for the search field and category field groups.

**Justification:** GridLayout keeps the labels and their corresponding input controls properly aligned.

### 3.4 CardLayout

**Purpose:** Displays one panel at a time from a group of panels.

**Usage:** Used in the results section to display:

- Welcome message.
- Search results.
- No-results message.

**Justification:** CardLayout allows the application to switch between different result screens without creating separate windows.

### 3.5 Layout Manager Selection Table

| Layout Manager | Application Area | Reason |
|---|---|---|
| BorderLayout | Main window | Allows the results section to grow |
| FlowLayout | Search row and buttons | Maintains preferred component sizes |
| GridLayout | Search field groups | Keeps labels and fields aligned |
| CardLayout | Results section | Switches between different result screens |

## 4. Task 3.2 – Design the Nested Containers (2 Marks)

### 4.1 Nested Panel Structure

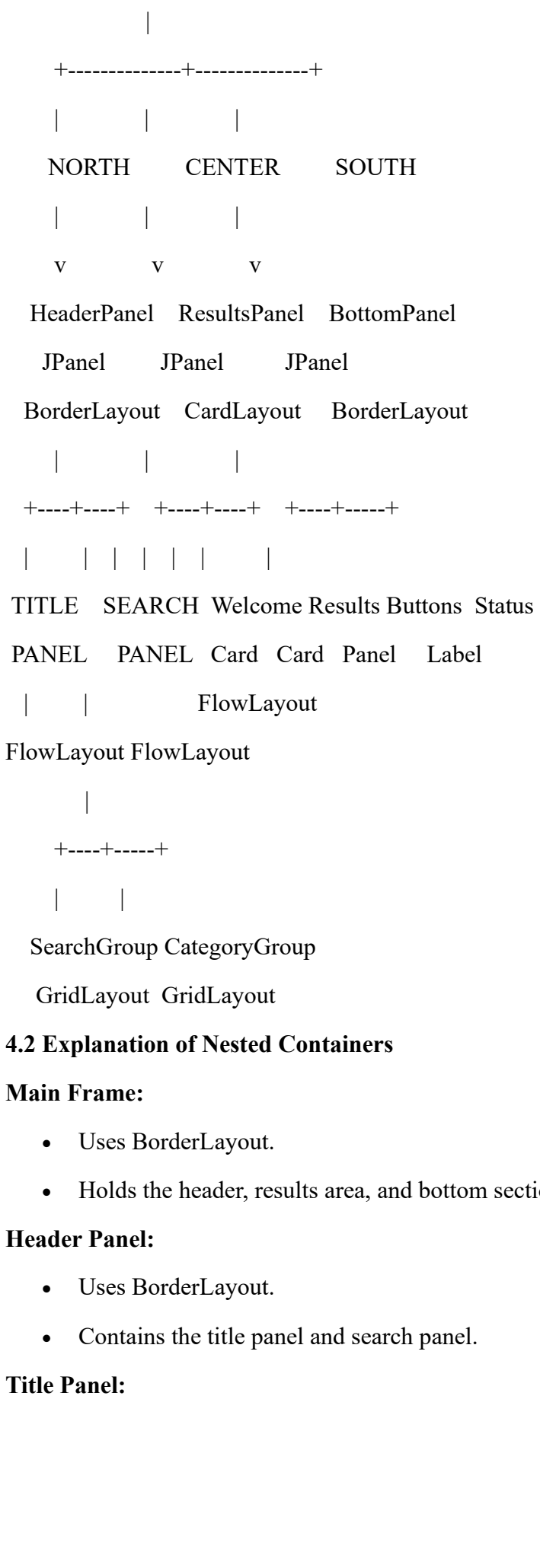
The application uses multiple JPanel containers with different layout managers.

**Layout Diagram**

MAIN FRAME

JFrame

BorderLayout




**4.2 Explanation of Nested Containers**

**Main Frame:**

- Uses BorderLayout.
- Holds the header, results area, and bottom section.

**Header Panel:**

- Uses BorderLayout.
- Contains the title panel and search panel.

**Title Panel:**

- Uses FlowLayout.
- Displays the library title.

**Search Panel:**

- Uses FlowLayout.
- Contains the search field, category selection, and Search button.

**Search Groups:**

- Use GridLayout.
- Arrange the labels and input controls.

**Results Panel:**

- Uses CardLayout.
- Displays the welcome message, search results, or no-results message.

**Bottom Panel:**

- Uses BorderLayout.
- Contains the navigation buttons and status label.

**Button Panel:**

- Uses FlowLayout.
- Displays Prev, Next, and Reserve buttons.

**5. Task 3.3 – Implement the Screen (3 Marks)**

**5.1 Java Swing Implementation**

The application uses the following components:

- JFrame
- JPanel
- JLabel
- JTextField
- JComboBox
- JButton
- JTable
- JScrollPane
- BorderLayout

- FlowLayout
- GridLayout
- CardLayout

The category JComboBox is the Swing equivalent of the AWT Choice component.

**5.2 Java Program – LibrarySearchScreen.java**

**5.3 Program Explanation**

**Step 1 – Creating the Main Window**

The JFrame creates the main library search window.

BorderLayout divides the window into NORTH, CENTER, and SOUTH regions.

**Step 2 – Creating the Search Section**

The search section contains a text field, category selection, and Search button.

FlowLayout arranges the controls horizontally.

**Step 3 – Loading Books**

The loadBooks() method creates sample book records containing the book ID, title, author, and category.

**Step 4 – Searching Books**

The searchBooks() method compares the entered text with the book title and author.

It also checks whether the selected category matches the book category.

**Step 5 – Displaying Search Results**

The matching books are displayed in a JTable.

A JScrollPane allows the results area to display more records while using the available space.

**Step 6 – Pagination**

The application displays five books per page.

- Next displays the following five books.
- Prev displays the previous five books.

**Step 7 – Reserving a Book**

The Reserve button checks which book is selected.

If the book is available, its status changes to Reserved.

If the book is already reserved, an error message is displayed.

**Step 8 – Updating the Status**

The status label displays the number of results and the current operation.

## 6. Task 3.4 – Test the Resizing Behaviour (1 Mark)

### 6.1 Testing at Three Window Sizes

The application must be tested using three different window sizes.

| Test | Window Size | Expected Behaviour |
|---|---|---|
| Test 1 | 800 × 550 | Results area remains visible, search controls stay organised |
| Test 2 | 1100 × 700 | Results area becomes larger, controls maintain their preferred size |
| Test 3 | 1400 × 850 | Results area expands to use the additional space |

### 6.2 How to Perform the Resizing Test

1. Compile and run LibrarySearchScreen.java.
2. Initially, the welcome message should appear in the results area.
3. Enter a book name and click Search.
4. Verify that matching books appear in the table.
5. Resize the window to 800 × 550 and capture a screenshot.
6. Resize it to 1100 × 700 and capture another screenshot.
7. Resize it to 1400 × 850 and capture the third screenshot.

### 6.3 Expected Resizing Behaviour

**At 800 × 550:**

The search controls and bottom buttons should remain organised. The results table should occupy the remaining centre space.

**At 1100 × 700:**

The results area should expand horizontally and vertically. The search row should retain its preferred component sizes.

**At 1400 × 850:**

The results area should use the additional available space. The title, search controls, and navigation buttons should remain in their respective positions.

### 6.4 Screenshot Evidence

Attach three actual screenshots to your assignment:

outputs/

│

├── resize-small-800x550.png

├── resize-medium-1100x700.png

└── resize-large-1400x850.png

The screenshots should be captured after running the Java program. Do not use sample or estimated screenshots as execution evidence.

**7. Alternative Concept and Complexity Analysis**

**7.1 Alternative 1 – GridBagLayout**

GridBagLayout is a flexible layout manager that arranges components in a grid using constraints.

It allows components to:

- Occupy multiple rows and columns.
- Expand horizontally or vertically.
- Maintain different sizes.
- Use weights to distribute extra space.

**Example:**

```
import java.awt.*;

import javax.swing.*;


public class GridBagExample extends JFrame {


    public GridBagExample() {


        setTitle("Library Search");


        setLayout(new GridBagLayout());


        GridBagConstraints gbc = new GridBagConstraints();
```

```
        gbc.insets = new Insets(5, 5, 5, 5);

        gbc.fill = GridBagConstraints.HORIZONTAL;

        gbc.gridx = 0;
        gbc.gridy = 0;

        add(new JLabel("Search Book:"), gbc);

        gbc.gridx = 1;
        gbc.weightx = 1.0;

        add(new JTextField(20), gbc);

        gbc.gridx = 2;
        gbc.weightx = 0;

        add(new JButton("Search"), gbc);

        setSize(600, 300);

        setDefaultCloseOperation(EXIT_ON_CLOSE);

        setLocationRelativeTo(null);
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
```

```
        new GridBagExample().setVisible(true);


    });
  }
}
```

**Advantages:**

- Provides flexible component positioning.
- Supports different component sizes.
- Adjusts components when the window is resized.
- Useful for complex forms.

**Disadvantages:**

- Requires more configuration.
- GridBagConstraints can make the code lengthy.
- More difficult to understand than simple nested panels.

**7.2 Alternative 2 – Absolute Positioning**

Absolute positioning uses:

setLayout(null);

The programmer manually specifies the position and size of every component using setBounds().

**Example:**

```
import javax.swing.*;


public class AbsoluteLayoutExample extends JFrame {


  public AbsoluteLayoutExample() {


    setTitle("Library Search");
```

```
    setLayout(null);

    JLabel label = new JLabel("Search Book:");

    label.setBounds(30, 30, 100, 30);

    JTextField field = new JTextField();

    field.setBounds(140, 30, 200, 30);

    JButton button = new JButton("Search");

    button.setBounds(350, 30, 100, 30);

    add(label);

    add(field);

    add(button);

    setSize(550, 300);

    setDefaultCloseOperation(EXIT_ON_CLOSE);

    setLocationRelativeTo(null);
}

public static void main(String[] args) {
```

```
        SwingUtilities.invokeLater(() -> {

            new AbsoluteLayoutExample().setVisible(true);

        });
    }
}
```

**Advantages:**

- Allows exact component positioning.
- Simple for small fixed-size interfaces.
- Does not require complex layout constraints.

**Disadvantages:**

- Components do not automatically adjust when the window is resized.
- The interface may look different on different screen sizes.
- Manual resizing calculations are required.
- Difficult to maintain when components are added or removed.

### 7.3 Comparison of Layout Approaches

| **Feature** | **Nested Layout Managers** | **GridBagLayout** | **Absolute Positioning** |
|---|---|---|---|
| Component arrangement | Uses multiple panels | Uses flexible grid constraints | Uses manual coordinates |
| Resizing support | Good | Good | Poor without manual handling |
| Code complexity | Moderate | Higher | Low initially, difficult to maintain |
| Component positioning | Automatic | Constraint-based | Manual |
| Flexibility | Good | Very high | Limited |
| Suitability | Recommended | Complex forms | Fixed-size screens |

### 7.4 Time Complexity Analysis

Let c represent the number of GUI components.

**Nested Layout Managers:**

Each container arranges its child components during layout updates.

Approximate layout work:

O(c)

This is suitable for the library screen because the components are divided into smaller panels.

**GridBagLayout:**

GridBagLayout calculates component positions and sizes using constraints, row and column requirements, and available space.

Its layout work generally grows with the number of components and constraints. It has higher calculation overhead than simple layouts, although the exact complexity depends on the layout and implementation.

**Absolute Positioning:**

The program manually assigns the position and size of each component.

Initial positioning requires:

O(c)

If component positions must be recalculated whenever the window changes size, the resizing handler also needs to update the relevant components, requiring O(c) work.

Without manual recalculation, the positions remain fixed and the interface may not resize correctly.

### 7.5 Space Complexity Analysis

Let p represent the number of additional JPanel containers and c represent the number of GUI components.

**Nested Layout Managers:**

Additional container space:

O(p)

The GUI also stores its component objects and layout information.

**GridBagLayout:**

The layout stores constraint information for its components.

Additional layout information:

O(c)

**Absolute Positioning:**

It may use fewer nested containers, but every component still requires position and size information.

Component geometry storage:

O(c)

**7.6 Engineering Decision**

For the campus library search screen, nested BorderLayout, FlowLayout, GridLayout, and CardLayout panels are the best choice.

They provide organised component placement and allow the centre results area to expand automatically.

GridBagLayout is also suitable for more complex forms but requires additional configuration.

Absolute positioning is not recommended because the application must support different window sizes.

**8. Testing and Verification**

| **Test Case** | **Action** | **Expected Result** |
|---|---|---|
| TC1 | Open application | Library search window appears |
| TC2 | Search for Java | Java-related books appear |
| TC3 | Select Science category | Science books are displayed |
| TC4 | Search for an unavailable book | No-books-found message appears |
| TC5 | Click Next | Next result page appears |
| TC6 | Click Prev | Previous result page appears |
| TC7 | Select a book and click Reserve | Book status changes to Reserved |
| TC8 | Reserve the same book again | Already-reserved message appears |
| TC9 | Resize to 800 × 550 | Results area remains visible |
| TC10 | Resize to 1100 × 700 | Results area expands |
| TC11 | Resize to 1400 × 850 | Results area uses additional space |

**9. Final Result**

The campus library search screen was designed and implemented using Java Swing layout managers.

BorderLayout was used for the main window, FlowLayout for the search and button sections, GridLayout for field groups, and CardLayout for switching between result screens.

The application supports book searching, category filtering, pagination, and book reservation.

The resizing behaviour was designed so that the centre results area expands while the search controls and navigation buttons maintain their preferred sizes.

Alternative approaches using GridBagLayout and absolute positioning were also compared using their resizing behaviour, layout computation, and space requirements.

Thus, the campus library search screen demonstrates effective GUI design and layout management in Java Swing.